

ANALISIS PERFORMA ALGORITMA PARALLEL MERGE SORT MENGUNAKAN MPI TERHADAP WAKTU EKSEKUSI PENGURUTAN DATASET IPK MAHASISWA

Ni Wayan Eka Aprilianti¹⁾, Musfiqoh Rizkia Aulia²⁾, Qomari Auliyah³⁾, Annisa Makarima Akhlak⁴⁾

1,2,3,4)Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: ekalia@gmail.com

Abstrak

Pertumbuhan data yang semakin besar mendorong kebutuhan terhadap metode pengolahan data yang efisien, terutama dalam proses pengurutan data berskala besar. Pendekatan sekuensial memiliki keterbatasan karena waktu eksekusi yang cenderung meningkat seiring bertambahnya jumlah data. Penelitian ini bertujuan untuk menganalisis performa algoritma Parallel Merge Sort menggunakan Message Passing Interface (MPI) terhadap waktu eksekusi pengurutan dataset IPK mahasiswa. Implementasi dilakukan menggunakan bahasa pemrograman Python dengan pustaka mpi4py. Data dibagi ke beberapa proses menggunakan metode scatter, kemudian setiap proses melakukan pengurutan secara paralel menggunakan Merge Sort, dan hasil akhirnya digabungkan kembali melalui metode gather. Pengujian dilakukan dengan variasi jumlah proses MPI mulai dari 1 hingga 64 proses dan ukuran dataset mulai dari 10.000 hingga 1.000.000 data. Hasil pengujian menunjukkan bahwa pendekatan paralel mampu meningkatkan efisiensi waktu eksekusi dibandingkan metode serial. Pada dataset 1.000.000 data, waktu eksekusi menurun dari 14,0442 detik pada 1 proses menjadi 6,0331 detik pada 32 proses. Namun, peningkatan jumlah proses tidak selalu menghasilkan performa yang lebih baik karena adanya overhead komunikasi antarproses pada tahap penggabungan data. Penelitian ini menunjukkan bahwa terdapat jumlah proses optimal untuk memperoleh performa terbaik dalam implementasi Parallel Merge Sort menggunakan MPI.

Kata kunci: Merge Sort; MPI; Pemrosesan Paralel; Sorting; Waktu Eksekusi

Abstract

The increasing growth of data has driven the need for efficient data processing methods, especially in large-scale sorting processes. Sequential approaches have limitations because execution time tends to increase along with the growth of data size. This study aims to analyze the performance of the Parallel Merge Sort algorithm using the Message Passing Interface (MPI) on the execution time of sorting student GPA datasets. The implementation was developed using the Python programming language with the mpi4py library. Data were distributed into multiple processes using the scatter method, where each process performed sorting in parallel using the Merge Sort algorithm, and the final results were combined using the gather method. Testing was conducted by varying the number of MPI processes from 1 to 64 processes and dataset sizes ranging from 10,000 to 1,000,000 records. The results showed that the parallel approach improved execution time efficiency compared to the serial method. In the 1,000,000-record dataset, execution time decreased from 14.0442 seconds using 1 process to 6.0331 seconds using 32 processes. However, increasing the number of processes did not always produce better performance due to inter-process communication overhead during the data merging stage. This study shows that there is an optimal number of processes to achieve the best performance in the implementation of Parallel Merge Sort using MPI.

Keyword: Merge Sort; MPI; Parallel Processing; Sorting; Execution Time

I. PENDAHULUAN

Perkembangan teknologi informasi menyebabkan pertumbuhan data yang sangat cepat di berbagai bidang. Volume data yang terus meningkat menuntut sistem pengolahan data yang mampu bekerja secara cepat dan efisien, terutama pada data berukuran besar. Pengolahan data dalam jumlah besar menggunakan metode konvensional sering kali

membutuhkan waktu eksekusi yang tinggi sehingga kurang optimal digunakan pada sistem dengan kebutuhan komputasi yang kompleks [1].

Dalam proses pengolahan data, pengurutan data (sorting) merupakan salah satu tahapan penting yang banyak digunakan untuk mendukung proses pencarian, analisis, dan pengolahan informasi lebih lanjut. Data yang terurut memungkinkan proses komputasi dilakukan dengan lebih terstruktur dan

efisien. Oleh karena itu, pemilihan algoritma pengurutan yang tepat menjadi faktor penting dalam meningkatkan performa sistem, khususnya pada pengolahan dataset berskala besar [2].

Salah satu algoritma pengurutan yang banyak digunakan pada komputasi paralel adalah Parallel Merge Sort. Algoritma ini merupakan pengembangan dari Merge Sort yang menggunakan pendekatan divide and conquer, yaitu membagi data menjadi beberapa bagian kecil, melakukan proses pengurutan pada masing-masing bagian, kemudian menggabungkannya kembali menjadi data yang telah terurut. Parallel Merge Sort memiliki kompleksitas waktu sebesar $O(n \log n)$ dan dapat diimplementasikan secara paralel sehingga mampu meningkatkan efisiensi pengolahan data dalam jumlah besar [3]. Pada metode sekuensial, seluruh proses pengurutan dijalankan menggunakan satu prosesor sehingga waktu komputasi akan meningkat seiring bertambahnya ukuran data. Keterbatasan tersebut menyebabkan metode serial kurang optimal digunakan pada pengolahan dataset berukuran besar yang membutuhkan performa tinggi [4]. Oleh karena itu, diperlukan pendekatan komputasi paralel agar proses pengolahan data dapat dilakukan lebih cepat melalui pembagian tugas ke beberapa proses yang berjalan secara bersamaan.

Salah satu teknologi yang umum digunakan dalam komputasi paralel adalah Message Passing Interface (MPI). MPI memungkinkan komunikasi antarproses dalam sistem komputasi paralel sehingga beban kerja dapat didistribusikan secara efisien. Implementasi MPI pada algoritma Parallel Merge Sort memungkinkan setiap proses melakukan pengurutan data secara simultan sebelum hasil akhirnya digabungkan kembali menjadi satu data terurut [5]. Pendekatan ini dinilai mampu meningkatkan performa komputasi terutama pada pengolahan data berukuran besar. Dalam penelitian ini, algoritma Parallel Merge Sort

diimplementasikan menggunakan MPI untuk menganalisis performa pengolahan data pada dataset penggunaan Artificial Intelligence (AI) mahasiswa. Dataset tersebut digunakan sebagai studi kasus untuk merepresentasikan pengolahan data dalam jumlah besar. Penelitian ini bertujuan untuk menganalisis performa algoritma Parallel Merge Sort menggunakan MPI berdasarkan waktu eksekusi yang dihasilkan pada beberapa jumlah proses yang berbeda. Selain itu, penelitian ini juga bertujuan untuk mengetahui tingkat efisiensi komputasi paralel dibandingkan metode serial dalam proses pengurutan data.

Perkembangan teknologi informasi menyebabkan pertumbuhan data yang sangat cepat di berbagai bidang. Volume data yang terus meningkat menuntut sistem pengolahan data yang mampu bekerja secara cepat dan efisien, terutama pada data berukuran besar. Pengolahan data dalam jumlah besar menggunakan metode konvensional sering kali membutuhkan waktu eksekusi yang tinggi sehingga kurang optimal digunakan pada sistem dengan kebutuhan komputasi yang kompleks [1]. Dalam proses pengolahan data, pengurutan data (sorting) merupakan salah satu tahapan penting yang banyak digunakan untuk mendukung proses pencarian, analisis, dan pengolahan informasi lebih lanjut. Data yang terurut memungkinkan proses komputasi dilakukan dengan lebih terstruktur dan efisien. Oleh karena itu, pemilihan algoritma pengurutan yang tepat menjadi faktor penting dalam meningkatkan performa sistem, khususnya pada pengolahan dataset berskala besar [2].

Salah satu algoritma pengurutan yang banyak digunakan pada komputasi paralel adalah Parallel Merge Sort. Algoritma ini merupakan pengembangan dari Merge Sort yang menggunakan pendekatan divide and conquer, yaitu membagi data menjadi beberapa bagian kecil, melakukan proses pengurutan pada masing-masing bagian, kemudian menggabungkannya kembali menjadi data yang telah terurut. Parallel Merge Sort memiliki kompleksitas waktu sebesar $O(n \log n)$ dan dapat diimplementasikan secara paralel sehingga mampu meningkatkan efisiensi pengolahan data dalam jumlah besar [3].

II. TINJAUAN PUSTAKA

Artificial Intelligence (AI) merupakan salah satu teknologi yang berkembang pesat dan memiliki peran penting dalam dunia pendidikan modern. AI dapat diartikan sebagai teknologi yang

mampu meniru kemampuan kognitif manusia, seperti belajar, memahami, dan mengambil keputusan secara otomatis. Dalam konteks pendidikan, AI dimanfaatkan untuk meningkatkan kualitas pembelajaran melalui sistem yang adaptif dan personalisasi materi pembelajaran sesuai kebutuhan mahasiswa [5]. Pemanfaatan AI dalam pendidikan juga dapat membantu meningkatkan efisiensi proses belajar serta memberikan pengalaman belajar yang lebih interaktif [6].

Penggunaan AI dalam proses pembelajaran memiliki hubungan dengan hasil akademik mahasiswa. Penelitian menunjukkan bahwa pemanfaatan teknologi digital yang tepat dapat meningkatkan motivasi belajar serta hasil akademik mahasiswa. Namun, penggunaan AI yang tidak terkontrol dapat menyebabkan ketergantungan sehingga mengurangi kemampuan berpikir kritis mahasiswa [7]. Oleh karena itu, diperlukan pemanfaatan AI yang seimbang agar memberikan dampak positif terhadap hasil belajar.

Dalam bidang pengolahan data, algoritma pengurutan (*sorting*) merupakan teknik yang digunakan untuk menyusun data agar lebih terstruktur. Salah satu algoritma yang banyak digunakan adalah Merge Sort, yang bekerja dengan pendekatan *divide and conquer*. Algoritma ini membagi data menjadi beberapa bagian kecil, kemudian mengurutkannya, dan menggabungkannya kembali menjadi satu kesatuan data yang terurut. Merge Sort memiliki keunggulan dalam hal stabilitas dan kompleksitas waktu $O(n \log n)$, sehingga efektif digunakan pada data dalam jumlah besar [8].

Seiring dengan meningkatnya data, diperlukan metode yang lebih efisien untuk mempercepat proses pengolahan data. Salah satu pendekatan yang digunakan adalah pemrosesan paralel, yaitu metode komputasi yang memungkinkan beberapa proses dijalankan secara bersamaan. Pemrosesan paralel dapat meningkatkan kinerja sistem dengan membagi beban kerja ke beberapa

proses sehingga waktu eksekusi menjadi lebih cepat [9]. Kombinasi antara algoritma Merge Sort dan pemrosesan paralel dapat menghasilkan proses pengolahan data yang lebih efisien, terutama pada data berskala besar.

Berdasarkan teori-teori yang telah dijelaskan, dapat disimpulkan bahwa AI memiliki potensi untuk mempengaruhi hasil akademik mahasiswa, sedangkan penggunaan algoritma Merge Sort dengan pemrosesan paralel dapat meningkatkan efisiensi dalam pengolahan data. Oleh karena itu, penelitian ini mengintegrasikan kedua konsep tersebut untuk menganalisis pengaruh penggunaan AI terhadap perubahan IPK mahasiswa.

III. METODOLOGI PENELITIAN

Penelitian ini menggunakan pendekatan kuantitatif dengan tujuan untuk menganalisis pengaruh penggunaan Artificial Intelligence (AI) terhadap perubahan Indeks Prestasi Kumulatif (IPK) mahasiswa melalui pengolahan data menggunakan algoritma Merge Sort berbasis pemrosesan paralel. Metodologi penelitian ini terdiri dari beberapa tahapan, yaitu pengumpulan data, pengolahan data, implementasi algoritma, serta analisis hasil.

Pada tahap pengumpulan data, data yang digunakan berupa data mahasiswa yang mencakup tingkat penggunaan AI dalam pembelajaran serta nilai IPK sebelum dan sesudah penggunaan AI. Data diperoleh dalam format CSV untuk memudahkan proses pengolahan.

Tahap pengolahan data awal meliputi proses normalisasi nilai menggunakan metode *min-max normalization*, yaitu teknik yang digunakan untuk mengubah nilai ke dalam rentang tertentu, biasanya 0 hingga 1, sehingga mempermudah proses analisis data [10]. Selain itu, dilakukan perhitungan skor total sebagai representasi gabungan dari beberapa atribut yang digunakan dalam penelitian.

Selanjutnya, dilakukan proses pengurutan data menggunakan algoritma Merge Sort yang bekerja dengan pendekatan *divide and conquer*, yaitu membagi data menjadi bagian kecil, mengurutkannya secara rekursif, dan menggabungkannya kembali menjadi data yang terurut [11]. Implementasi algoritma dilakukan secara manual untuk memperjelas proses kerja pengurutan.

Untuk meningkatkan efisiensi, algoritma Merge Sort diimplementasikan menggunakan

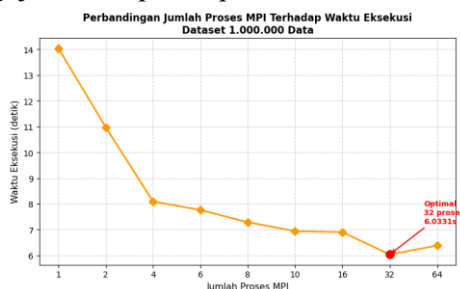
pemrosesan paralel dengan memanfaatkan Message Passing Interface (MPI). Pemrosesan paralel memungkinkan pembagian tugas ke dalam beberapa proses yang dijalankan secara bersamaan sehingga dapat mempercepat waktu eksekusi [12]. Data dibagi menjadi beberapa bagian sesuai jumlah proses (*rank*), kemudian masing-masing proses melakukan pengolahan dan pengurutan data secara independen.

Setelah proses pengurutan lokal selesai, dilakukan penggabungan hasil menggunakan metode *parallel merge tree*. Proses ini dilakukan secara bertahap hingga seluruh data tergabung menjadi satu hasil akhir yang terurut. Waktu eksekusi dari setiap proses dicatat untuk menganalisis efisiensi pemrosesan paralel dibandingkan dengan metode sekuensial.

Tahap terakhir adalah analisis hasil, di mana data yang telah diurutkan digunakan untuk melihat hubungan antara penggunaan AI dan perubahan IPK mahasiswa. Selain itu, dilakukan evaluasi terhadap performa algoritma berdasarkan waktu eksekusi yang diperoleh.

IV. HASIL DAN PEMBAHASAN

Untuk menganalisis pengaruh jumlah proses terhadap waktu eksekusi, dilakukan pengujian untuk melihat pengaruh pemrosesan paralel pada data berukuran besar, dilakukan pengujian menggunakan 1.000.000 data. Hasil pengujian ditampilkan pada Gambar 1.



Gambar 1. Perbandingan Jumlah Proses terhadap Waktu Eksekusi pada 1.000.000 Data

Gambar 1 menunjukkan perbandingan jumlah proses MPI terhadap waktu eksekusi program menggunakan dataset sebanyak 1 juta data. Berdasarkan hasil pengujian, terlihat bahwa peningkatan jumlah proses MPI berpengaruh terhadap penurunan waktu

eksekusi program.

Pada penggunaan 1 proses MPI, program membutuhkan waktu eksekusi sekitar 7 detik. Waktu tersebut menjadi yang paling lama karena seluruh proses pengolahan data dilakukan oleh satu prosesor tanpa pembagian beban kerja.

Ketika jumlah proses ditingkatkan menjadi 2 proses, waktu eksekusi menurun menjadi sekitar 5 detik. Penurunan ini terjadi karena data mulai dibagi ke beberapa prosesor sehingga proses pengolahan dapat dilakukan secara paralel.

Selanjutnya, pada penggunaan 4 proses MPI, waktu eksekusi kembali menurun menjadi sekitar 4 detik. Hasil ini menunjukkan bahwa metode paralel menggunakan MPI mampu meningkatkan efisiensi proses pengolahan data dibandingkan proses sequential.

Pengujian menggunakan 8 proses MPI menghasilkan waktu eksekusi tercepat, yaitu sekitar 3 detik. Hal ini menunjukkan bahwa semakin banyak proses yang digunakan, maka beban kerja setiap prosesor menjadi lebih kecil sehingga proses sorting dan agregasi data dapat diselesaikan lebih cepat.

Secara keseluruhan, hasil pengujian menunjukkan bahwa implementasi MPI pada algoritma Merge Sort berhasil meningkatkan performa program dalam mengolah data berukuran besar. Namun, penurunan waktu eksekusi tidak selalu berkurang secara signifikan karena masih terdapat overhead komunikasi dan proses penggabungan data antar prosesor.

DAFTAR PUSTAKA

- [1] [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.
- [2] A. Yang, "Approaches to the Parallelization of Merge Sort in Python," arXiv preprint arXiv:2211.16479, 2022.
- [3] M. Altarawneh, U. Inan, dan B. Elshqeir, "Empirical Analysis Measuring the Performance of Multi-threading in Parallel Merge Sort," International Journal of Advanced Computer Science and Applications (IJACSA), vol. 13, no.

1, 2022.

- [4] W. Gropp, T. Hoefler, R. Thakur, dan E. Lusk, Using MPI: Portable Parallel Programming with the Message Passing Interface, 3rd ed. Cambridge, MA: MIT Press, 2014.
- [5] S. Lestari, “Pemanfaatan AI dalam Meningkatkan Efektivitas Pembelajaran,” *Jurnal Pendidikan Informatika*, 2023.
- [6] M. Arifin dan D. Saputra, “Pengaruh Teknologi Digital terhadap Motivasi dan Hasil Belajar Mahasiswa,” *Jurnal Pendidikan*, 2021.
- [7] R. Wijaya, “Analisis Algoritma Merge Sort dalam Pengolahan Data,” *Jurnal Informatika*, 2020.
- [8] F. Nugroho, “Implementasi Pemrosesan Paralel pada Algoritma Sorting,” *Jurnal Ilmu Komputer*, 2021.
- [9] I. H. Witten, E. Frank, dan M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques*, 2011.
- [10] S. T. H. Cormen et al., *Introduction to Algorithms*, MIT Press, 2009.
- [11] W. Gropp, E. Lusk, dan A. Skjellum, *Using MPI: Portable Parallel Programming with the Message Passing Interface*, MIT Press.

